



botdc

Bot de Discord que segue em tempo real o que está tocando no seu Spotify

Documentação funcional

Você troca a música no celular e o bot troca na call, começando no mesmo ponto em que o seu Spotify está. Este documento lista tudo o que ele faz: comandos, detecção de faixa, reprodução, desempenho, tolerância a falhas e configuração.

Gerado em 24/07/2026

1. Comandos

Qualquer pessoa pode usar o bot, no servidor dela, sem login e sem token. Cada servidor tem uma sessão própria, seguindo o Spotify de quem rodou `/vincular` — o "driver". A mesma pessoa pode comandar vários servidores ao mesmo tempo.

Comando	O que faz
<code>/vincular [canal]</code>	Entra no canal de voz e passa a seguir o SEU Spotify. Sem argumento, usa o canal em que você está. Se já houver música tocando, começa na hora.
<code>/desvincular</code>	Para de seguir, sai do canal de voz e limpa a fila.
<code>/modo follow queue</code>	Define o que acontece quando você troca de música no Spotify. Em follow a troca é imediata; em queue a faixa nova espera a atual terminar.
<code>/agora</code>	Mostra faixa, artista, álbum, capa, barra de progresso, qual vídeo foi escolhido no YouTube, o modo ativo e a fonte do dado (presence ou API).
<code>/pular</code>	Pula a faixa atual. Em modo queue, avança para a próxima da fila.
<code>/fila</code>	Lista as faixas enfileiradas (até 15, com contagem do restante).
<code>/rematch</code>	Achou o vídeo errado? Descarta o match guardado em cache e procura de novo.
<code>/sr <música></code>	Pede uma música sem depender do Spotify: busca por nome ou aceita um link. Se o bot não estiver em canal nenhum, já entra no seu e começa a tocar.
<code>/ajuda</code>	Lista todos os comandos com suas descrições e explica, dentro do Discord, como ligar o Spotify à conta. A lista é montada a partir das próprias definições dos comandos, então nunca fica defasada.

Quem pode usar o quê

Comando	Quem pode
<code>/sr</code> , <code>/vincular</code> , <code>/agora</code> , <code>/fila</code> , <code>/pular</code> , <code>/ajuda</code>	Qualquer pessoa do servidor
<code>/desvincular</code> , <code>/modo</code> , <code>/rematch</code>	O driver, ou quem tem Gerenciar Servidor

Assumir o lugar de quem já está comandando exige Gerenciar Servidor: sem isso, qualquer um derrubaria a sessão alheia no meio da música. Quando o canal de voz esvazia, a sessão encerra sozinha.

Modo jukebox: sem Spotify

- Não é preciso vincular nada. O `/sr` pede música por nome ou por link, e se o bot não estiver em canal nenhum, ele já entra no seu e começa a tocar.
- Sem ninguém sendo seguido, não há driver a proteger: a fila é coletiva e os controles ficam abertos a todo mundo.
- Um pedido nunca corta o que está tocando — quem pediu antes ouve inteiro. O `/fila` mostra quem pediu cada faixa.
- Link de playlist traz só o vídeo apontado, senão um único pedido viraria centenas de faixas.

- Dá para usar /sr numa sessão já vinculada ao Spotify, mas em modo follow a próxima troca interrompe o pedido. O bot avisa disso na resposta e sugere /modo queue, onde as duas fontes convivem na mesma fila.

2. Detecção do que está tocando

Duas fontes independentes alimentam o bot, e elas se cobrem: se uma não estiver disponível, a outra sustenta o funcionamento sozinha.

Presence do Discord

- Chega por push, em cerca de 1 segundo, sem nenhuma consulta de rede da nossa parte.
- Traz id da faixa, título, artista, álbum e os horários de início e fim.
- Depende de o Spotify estar conectado à sua conta do Discord e de "Exibir o Spotify como seu status" estar ligado.
- Não informa progresso exato nem distingue pausa de "parou de compartilhar".

Web API do Spotify (opcional, uma conta só)

- Consulta periódica (padrão a cada 3 segundos) ao endpoint de faixa atual.
- Sabe o progresso em milissegundos, se está pausado e em qual dispositivo.
- Também lê a fila, que é o que viabiliza o prefetch descrito na seção 4.
- Respeita limite de requisições: em resposta 429, recua pelo tempo que o Spotify pedir.
- Vale apenas para a conta de OWNER_USER_ID, porque o token do .env pertence a uma pessoa só. Todos os demais rodam pela presence.

O que muda sem a Web API

Recurso	Só presence	Presence + API
Trocar de faixa junto	Sim	Sim
Sincronizar a posição	Sim	Sim
Espelhar pausa e retomada	Não	Sim
Prefetch (troca instantânea)	Não	Sim

Como as duas se combinam

- A API é a fonte da verdade para estado e posição; a presence funciona como gatilho rápido.
- Quando a presence acusa uma faixa diferente, o bot já começa a resolver com esses dados em vez de esperar a ida e volta da API. São cerca de 300 ms a menos de silêncio.
- A consulta seguinte refina progresso e estado. A deduplicação por id da faixa garante que a música não toque duas vezes.
- Ao iniciar, o bot lê a presence que já estava no ar. Sem isso, subir com música tocando não detectaria nada até a próxima troca.
- Podcasts e episódios são ignorados: só faixas de música.

3. Reprodução no Discord

A API do Spotify não entrega áudio em nenhum endpoint nem em nenhum plano. O bot usa apenas os metadados e reprocura a faixa no YouTube, de onde sai o áudio de fato.

Escolha do vídeo

- Busca cinco candidatos e ranqueia por proximidade de duração com a faixa do Spotify, que é o sinal mais confiável de ser a mesma gravação.
- Dá preferência a canais "- Topic", que são uploads automáticos da gravadora.
- Penaliza títulos com marcadores de versão diferente (live, cover, karaokê, sped up, nightcore e afins), a menos que a própria faixa os tenha no nome.

Áudio

- O ffmpeg transcodifica para ogg/opus a 128 kbps, 48 kHz, estéreo, que é o formato que o Discord aceita direto.
- Com SYNC_POSITION ligado, a música começa no mesmo ponto em que o seu Spotify está, já somando o tempo gasto na busca. Você não ouve a faixa atrasada, apenas espera menos.
- O bot entra no canal em modo surdo, sem consumir áudio de ninguém.

Cartão "Reproduzindo agora"

- A cada troca de música, um cartão é publicado no canal de texto onde /vincular foi usado: capa do álbum, faixa com link para o Spotify, quem está sendo seguido, canal de voz, contadores e barra de progresso.
- A barra é uma imagem PNG gerada na hora, porque embed do Discord não renderiza gradiente nem alça circular. O preenchimento vai do preto ao branco e termina sempre claro na alça.
- Não usa biblioteca gráfica: os pixels são rasterizados à mão e o PNG é codificado com o zlib do próprio Node. Os dígitos saem de uma fonte 5x7 desenhada no próprio arquivo.
- O cartão sai depois que o áudio começa, então já mostra qual vídeo foi escolhido e em que ponto o bot entrou.
- O cartão anterior é apagado a cada troca, para o canal não virar um mural. Desliga com ANNOUNCE_TRACKS=false.

Modos e espelhamento

Situação no Spotify	O que o bot faz
Troca de faixa, modo follow	Interrompe o que estava tocando e toca a faixa nova, sincronizada na posição.
Troca de faixa, modo queue	Enfileira. A faixa da fila começa do início quando chega a vez dela, e não no ponto em que entrou.
Pausa	Pausa a reprodução no canal de voz.
Retoma	Retoma de onde parou.
Para de tocar	Encerra o áudio e limpa o status do bot.

Situação no Spotify	O que o bot faz
Qualquer faixa nova	O status do bot passa a exibir "Ouvindo <faixa> - <artista>".

4. Desempenho

Resolver uma faixa custa duas chamadas ao yt-dlp: a busca e a extração da URL de áudio. Três mecanismos evitam pagar esse custo na frente do usuário.

Cache em duas camadas

Chave	Guarda	Vence
match:<id spotify>	Id do vídeo no YouTube	Nunca; a escolha não muda
stream:<id vídeo>	URL assinada e cabeçalhos	Junto com a assinatura, menos 15 min de margem

A margem de 15 minutos existe para a URL não vencer no meio de uma faixa longa. O arquivo fica em `cache/resolve.json`, com escrita atômica, teto de 2000 entradas e descarte do menos usado.

Prefetch

O cache sozinho só ajuda na segunda vez. Para a primeira também ser instantânea, o bot consulta a fila do Spotify cinco segundos depois de cada troca e resolve as duas próximas faixas enquanto a atual toca. Ouvindo álbum ou playlist na ordem, a troca já está pronta. A espera de cinco segundos é proposital: logo após a troca, a faixa atual ainda está resolvendo, e não vale competir com ela por rede e processamento.

Binário onedir

No Windows o bot usa o build onedir do yt-dlp em vez do executável único. O onefile é um pacote que se reextrai a cada execução, custando cerca de 1,6 s de inicialização toda vez; já descompactado, o mesmo binário inicia em 0,4 s. Como são duas chamadas por faixa, isso sozinho cortou aproximadamente 2,4 s do atraso.

Resultado medido

Situação	Tempo até tocar
Sequência normal de playlist ou álbum (pré-buscada)	aprox. 1 ms
Faixa já tocada antes, URL vencida	aprox. 1,6 s
Faixa nova e fora da fila (pulo aleatório)	aprox. 3,4 s

O piso é o próprio yt-dlp, que gasta cerca de 1,6 s por chamada e não tem modo servidor. Para ficar abaixo disso seria preciso trocar a fonte de áudio.

5. Tolerância a falhas

- URL de áudio vencida ou recusada: o ffmpeg morre em segundos sem produzir som. O bot detecta, descarta a URL do cache e tenta uma vez com uma nova.
- Vídeo removido, privado ou bloqueado na região: o vínculo entre a faixa e aquele vídeo é desfeito e a busca, refeita.
- Troca rápida de música: resoluções que ficaram obsoletas no meio do caminho são descartadas, então a faixa antiga nunca começa a tocar por cima da nova.
- Queda na conexão de voz: se for apenas mudança de região, o bot reata sozinho; se for perda real, sai do canal de forma limpa.
- Limite de requisições do Spotify: recuo pelo tempo indicado, sem derrubar o bot.
- Falha de rede na consulta: recuo curto para não inundar o log, e a consulta seguinte segue normalmente.
- Cache corrompido ou ausente: começa vazio, sem impedir a inicialização.
- Fila indisponível: o prefetch é best-effort, e a faixa apenas resolve na hora de tocar.
- Encerramento: o cache é gravado antes de sair, para os matches da sessão não se perderem.

6. Configuração

Tudo vem do arquivo .env. Variáveis obrigatórias ausentes são apontadas pelo nome na inicialização.

Variável	Para que serve
DISCORD_TOKEN	Token do bot. Obrigatório.
DISCORD_CLIENT_ID	Application ID, usado no registro dos comandos. Obrigatório.
DISCORD_GUILD_ID	Servidor onde registrar os comandos. Deixe VAZIO para registro global, que é o necessário para o bot funcionar em servidores de outras pessoas. Propaga em até 1 h.
OWNER_USER_ID	Opcional. Identifica de quem é a conta do Spotify configurada abaixo; só essa pessoa ganha os extras da Web API.
SPOTIFY_CLIENT_ID	Opcional. Credencial da Web API. Sem ela, todos rodam pela presence.
SPOTIFY_CLIENT_SECRET	Credencial da Web API.
SPOTIFY_REFRESH_TOKEN	Gerado por npm run login:spotify. Não expira.
SPOTIFY_REDIRECT_URI	Padrão http://127.0.0.1:8888/callback. O Spotify exige loopback e não aceita mais localhost.
POLL_INTERVAL_MS	Intervalo de consulta à Web API. Padrão 3000.
DEFAULT_MODE	follow ou queue. Padrão follow.
SYNC_POSITION	Começar no mesmo ponto do seu Spotify. Padrão ligado.
ANNOUNCE_TRACKS	Publicar o cartão "Reproduzindo agora" a cada troca. Padrão ligado.
LOG_LEVEL	debug, info, warn ou error. Em debug mostra ranking dos candidatos, acertos de cache e a saída do ffmpeg.

7. Comandos de terminal

Comando	O que faz
<code>npm start</code>	Liga o bot.
<code>npm run setup:yt-dlp</code>	Baixa ou atualiza o yt-dlp. Primeira coisa a tentar quando o YouTube quebrar a extração.
<code>npm run login:spotify</code>	Faz o login no Spotify e imprime o refresh token para colar no <code>.env</code> .
<code>npm run deploy:commands</code>	Registra os comandos de barra. Com <code>DISCORD_GUILD_ID</code> vazio, registra globalmente e limpa registros de servidor sobrando, que é o que faz os comandos aparecerem duplicados.
<code>npm run check</code>	Checagem offline: binários, carregamento dos módulos, montagem dos comandos, cache e lógica de detecção. Não precisa de credenciais.
<code>npm run test:audio</code>	Diagnóstico da cadeia de áudio isolada do Discord. Aceita a busca e um ponto de início em segundos.
<code>npm run docs:pdf</code>	Gera este documento.

8. Limitações conhecidas

- A API do Spotify não entrega áudio. Reprocurar a faixa no YouTube é o que torna a reprodução possível, e também o que viola os termos de uso daquele serviço. Para uso pessoal em servidor próprio é o caminho padrão; para um bot público, não é.
- O match nem sempre é perfeito: a escolha do vídeo é heurística. Confira com `/agora` e corrija com `/rematch`.
- Um canal de voz por servidor, seguindo uma pessoa por vez. Vários servidores em paralelo, sim.
- Web API do Spotify: 25 usuários. Um app em modo de desenvolvimento exige cadastrar cada pessoa manualmente no painel, e a liberação de cota passa por revisão da Spotify, que costuma recusar exatamente este tipo de uso. Por isso a Web API vale para uma conta só e todo o resto roda pela `presence`.
- Discord: 100 servidores sem verificação. Verificar exige justificar o `Presence Intent`, que é privilegiado e difícil de aprovar para um bot que lê o que os outros ouvem.
- Cada servidor ativo custa uma conexão de voz e um `ffmpeg` na máquina que hospeda.
- Faixa nova fora da fila ainda custa cerca de 3,4 s, limitados pelo `yt-dlp`.
- Se o YouTube apertar o cerco na extração, atualizar o `yt-dlp` resolve na maioria das vezes. Essa é a manutenção recorrente deste tipo de bot.